

Lipahuru Payment Gateway — Merchant Integration Guide

Version: 1.0 **Audience:** Merchant developers, backend engineers, solution architects **Currency:** TZS (Tanzania Shillings)

1. Overview

Lipahuru is a payment gateway that lets your application accept and send mobile money payments across Tanzanian networks through a **single REST API**.

You integrate once with Lipahuru. We handle routing to the underlying mobile money providers internally. You specify the target network using `providerCode` — you never need to integrate with individual telcos directly.

Supported networks (`providerCode`)

Code	Network
<code>`YAS`</code>	Yas (Tigo)
<code>`VODACOM`</code>	Vodacom M-Pesa
<code>`HALOTEL`</code>	Halotel
<code>`AIRTEL`</code>	Airtel Money

Payment types

Type	Description
<code>**C2B push collection**</code>	Request payment from a customer's mobile wallet
<code>**B2C disbursement**</code>	Send funds to a customer's mobile wallet
<code>**Refunds**</code>	Reverse a successful collection (partial or full)

Environments

Environment	Base URL
UAT	<code>`https://uat-api.lipahuru.com`</code> *(replace with your UAT host)*
Production	<code>`https://api.lipahuru.com`</code> *(replace with your production host)*

Use separate credentials and callback URLs for UAT and production.

2. Getting started

2.1 Onboarding

Contact Lipahuru to register your business. After approval you will receive:

Credential	Purpose	Shown again?
`client_id`	OAuth2 client identifier	No — store securely
`client_secret`	OAuth2 token + HMAC request signing	**Once only**

Your merchant account must be **ACTIVE** before payment APIs will work.

2.2 Integration checklist

- ☐ Store credentials in a secrets manager (never in source code)
- ☐ Implement OAuth token retrieval
- ☐ Implement HMAC request signing with `client\_secret`
- ☐ Expose an HTTPS callback endpoint
- ☐ Handle idempotency and duplicate callbacks safely
- ☐ Test in UAT before going live

3. Authentication

Every payment API call uses **two headers**:

1. **Bearer token** — obtained via OAuth2 client credentials
2. **HMAC signature** — signed with your `client_secret`

3.1 Obtain access token

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id={client_id}&client_secret={client_secret}
```

Success response:

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Tokens expire after **900 seconds (15 minutes)**. Cache the token and refresh before expiry.

3.2 Required headers (all payment APIs)

Header	Required	Description
`Authorization`	Yes	`Bearer {access_token}`
`X-Signature`	Yes	Base64 HMAC-SHA256 signature (see §3.3)
`X-Idempotency-Key`	Payment creation only	Unique key per payment attempt

3.3 Request signature

Build the **canonical string** (each value on its own line):

```
{HTTP_METHOD}
{REQUEST_PATH}
{content_sha256}
```

Where `content_sha256 = Base64( SHA256( raw_request_body ) )`. For **GET** requests the body is empty — hash the empty string.

Example:

```
POST
/api/v1/payments/collections/push
rSBedSEdHQ8w/MunXoV5jCQ/HKS6QuVjJSfJXQwa4DY=
```

Compute the signature:

```
X-Signature = Base64( HMAC-SHA256( canonical_string, client_secret ) )
```

3.4 PHP signing example

```
function signRequest(
    string $method,
    string $path,
    string $body,
    string $clientSecret,
    string $accessToken,
): array {
    $contentSha256 = base64_encode(hash('sha256', $body, true));

    $canonical = implode("\n", [
        strtoupper($method),
        $path,
        $contentSha256,
    ]);

    $signature = base64_encode(hash_hmac('sha256', $canonical, $clientSecret, true));

    return [
        'Authorization' => 'Bearer ' . $accessToken,
        'X-Signature' => $signature,
    ];
}
```

4. Response format

All API responses use this envelope:

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Request processed successfully",
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "timestamp": "2026-07-03T14:30:05+03:00",
  "data": { }
}
```

Field	Description
-------	-------------

`status`	`SUCCESS` or `FAILED`
`code`	Machine-readable result code (see §10)
`message`	Human-readable description
`requestId`	Echo of your `X-Request-Id`
`data`	Response payload (null on failure)

5. Payment APIs

5.1 C2B push collection

Initiate a payment request to a customer's mobile wallet. The customer approves on their phone. Final result is delivered asynchronously via callback.

```
POST /api/v1/payments/collections/push
```

Request body:

```
{
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "providerCode": "VODACOM",
  "amount": 10000.00,
  "currency": "TZS",
  "msisdn": "255754123456",
  "reference": "INV-1001",
  "callbackUrl": "https://your-app.com/webhooks/lipahuru",
  "narration": "Invoice INV-1001"
}
```

Field rules:

Field	Required	Rules
`requestId`	Yes	UUID v4, unique per request
`providerCode`	Yes	`YAS`, `VODACOM`, `HALOTEL`, or `AIRTEL`
`amount`	Yes	Minimum 100.00, max 4 decimal places
`currency`	No	Default `TZS`
`msisdn`	Yes	International format: `255` + 9 digits
`reference`	Yes	Your business reference (invoice, order ID)
`callbackUrl`	No	HTTPS URL; uses merchant default if omitted
`narration`	No	Payment description shown to customer

Success response (acknowledgement — not final payment):

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "message": "Request processed successfully",
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "timestamp": "2026-07-03T14:34:41+03:00",
  "data": {
    "transactionId": "TXN-A1B2C3D4E5F6G7H8",
    "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
    "reference": "INV-1001",
    "operation": "C2B_USSD_PUSH",
    "transactionStatus": "ACKNOWLEDGED",
    "providerCode": "VODACOM",
    "amount": "10000.0000",
    "currency": "TZS",
    "msisdn": "255754123456"
  }
}
```

****Important:**** `ACKNOWLEDGED` means the push was sent — not that the customer paid. Wait for the webhook callback or poll the status endpoint for the final result.

5.2 B2C disbursement

Send funds from your disbursement wallet to a customer's mobile wallet.

```
POST /api/v1/payments/disbursements
```

Request body:

```
{
  "requestId": "6e3c1d3d-6b96-4e08-8a7d-c4968ea6905a",
  "providerCode": "YAS",
  "amount": 25000.00,
  "currency": "TZS",
  "msisdn": "255713123456",
  "reference": "PAYOUT-1001",
  "callbackUrl": "https://your-app.com/webhooks/lipahuru",
  "narration": "Salary payment"
}
```

Success response:

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "data": {
    "transactionId": "TXN-X9Y8Z7W6V5U4T3S2",
    "transactionStatus": "FUNDS_RESERVED",
    "providerCode": "YAS",
    "amount": "25000.0000",
    "currency": "TZS"
  }
}
```

FUNDS_RESERVED means funds were held in your disbursement wallet. Final success or failure comes via callback.

5.3 Query transaction status

```
GET /api/v1/payments/{transactionId}
```

Use the transactionId returned when you created the payment. Requires signed headers (empty body).

Success response:

```
{
  "status": "SUCCESS",
  "code": "PGW-0000",
  "data": {
    "transactionId": "TXN-A1B2C3D4E5F6G7H8",
    "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
    "reference": "INV-1001",
    "operation": "C2B_USSD_PUSH",
    "transactionStatus": "SUCCESS",
    "providerCode": "VODACOM",
    "amount": "10000.0000",
    "currency": "TZS",
    "msisdn": "255754123456",
    "providerTransactionId": "919994765",
    "providerReceiptNo": "5HN31LXJQAX",
    "finalizedAt": "2026-07-03T14:38:57+03:00"
  }
}
```

5.4 Transaction history

```
GET /api/v1/transactions?perPage=25&page=1&status=SUCCESS&from=2026-07-01&to=2026-07-31
```

Query param	Description
`perPage`	Results per page (default 25)
`page`	Page number
`status`	Filter by transaction status
`from` / `to`	Date range filter
`reference`	Filter by your business reference

5.5 Refund

Refund a successful C2B collection.

```
POST /api/v1/payments/{transactionId}/refunds
```

Request body:

```
{
  "requestId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "amount": 5000.00,
  "reason": "Customer returned goods"
}
```

Omit amount for a full refund.

5.6 Wallet balances

```
GET /api/v1/wallets
GET /api/v1/wallets/{providerCode}
```

Response:

```
{
  "status": "SUCCESS",
  "data": {
    "walletId": 12,
    "walletType": "COLLECTION_LEAF",
    "providerCode": "VODACOM",
    "currency": "TZS",
    "available": "150000.0000",
    "reserved": "0.0000",
    "total": "150000.0000"
  }
}
```

6. Transaction lifecycle

RECEIVED → AUTHENTICATED → VALIDATED → [FUNDS_RESERVED] → ACKNOWLEDGED
→ PENDING_FINAL → SUCCESS | FAILED | REVERSED

Status	Meaning
`ACKNOWLEDGED`	Request accepted, processing started
`PENDING_FINAL`	Awaiting customer action or provider confirmation
`FUNDS_RESERVED`	B2C: amount held in disbursement wallet
`SUCCESS`	Payment completed successfully
`FAILED`	Payment failed
`REVERSED`	Refund completed against original transaction

Your integration should treat only **`SUCCESS`** and **`FAILED`** as terminal states for business logic.

7. Webhooks (callbacks)

When a payment reaches a final state, Lipahuru sends an HTTP POST to your callback URL.

7.1 Your endpoint requirements

Requirement	Value
Protocol	HTTPS only
Method	POST
Content-Type	`application/json`
Response	Return HTTP **200** immediately
Idempotency	Handle duplicate callbacks for the same `transactionId`

7.2 Callback payload

```
{
  "event": "PAYMENT_FINALIZED",
  "transactionId": "TXN-A1B2C3D4E5F6G7H8",
  "requestId": "3cf73c52-d6b0-4906-8b5d-f9a4f82b5011",
  "reference": "INV-1001",
  "status": "SUCCESS",
}
```

```

"operation": "C2B_USSD_PUSH",
"amount": "10000.0000",
"currency": "TZS",
"msisdn": "255754123456",
"providerTransactionId": "919994765",
"providerReceiptNo": "5HN31LXJQAX",
"failureCode": null,
"failureMessage": null,
"finalizedAt": "2026-07-03T14:38:57+03:00",
"timestamp": "2026-07-03T14:38:58+03:00"
}

```

7.3 Callback delivery

Lipahuru sends callbacks as a plain JSON POST with Content-Type: application/json. No signature headers are included — secure your callback URL (HTTPS, firewall, IP allowlist if needed).

7.4 Your response

Return HTTP 200 immediately:

```

{
  "status": "RECEIVED",
  "message": "Callback accepted"
}

```

Process the payment update asynchronously in your application — do not perform slow operations before responding.

7.5 Retries

If your endpoint does not return HTTP 200 within 30 seconds, Lipahuru retries with exponential backoff:

Attempt	Delay after failure
1	1 minute
2	5 minutes
3	15 minutes
4	1 hour
5	6 hours
6+	24 hours

Maximum **10 attempts**. Always handle duplicate callbacks idempotently.

8. Idempotency

Payment creation endpoints require X-Idempotency-Key.

Scenario	Behaviour
Same key + same payload	Returns the original result
Same key + different payload	Rejected with `PGW-1004`
Same `requestId` already used	Rejected with `PGW-1004`

Best practice: Generate a new UUID for each unique payment attempt. Reuse the same key only when retrying after a network timeout with the identical payload.

9. End-to-end flows

C2B collection

1. Your server → POST /oauth/token (get bearer token)
2. Your server → POST /api/v1/payments/collections/push (signed)
3. Lipahuru → 200 ACKNOWLEDGED + transactionId
4. Customer → Approves payment on mobile
5. Lipahuru → POST your-callback-url (PAYMENT_FINALIZED)
6. Your server → 200 {"status":"RECEIVED"}

If no callback arrives within 2 minutes, poll GET /api/v1/payments/{transactionId}.

B2C disbursement

1. Your server → POST /oauth/token
2. Your server → POST /api/v1/payments/disbursements (signed)
3. Lipahuru → 200 FUNDS_RESERVED + transactionId
4. Lipahuru → Processes disbursement with provider
5. Lipahuru → POST your-callback-url (SUCCESS or FAILED)
6. Your server → 200 {"status":"RECEIVED"}

10. Error codes

Code	HTTP	Meaning
`PGW-0000`	200	Success
`PGW-1001`	400	Invalid request payload
`PGW-1002`	400	Unsupported or disabled `providerCode`
`PGW-1003`	400	Invalid MSISDN format
`PGW-1004`	409	Duplicate `requestId` or idempotency conflict
`PGW-1005`	400	Amount exceeds merchant limits
`PGW-1006`	400	Insufficient disbursement wallet balance
`PGW-1007`	401	Authentication failed
`PGW-1008`	401	Signature validation failed
`PGW-1009`	401	Replay protection failed (nonce/timestamp)
`PGW-1010`	404	Transaction not found
`PGW-1099`	502	General processing error

Example error response:

```
{  
  "status": "FAILED",
```

```

"code": "PGW-1006",
"message": "Insufficient available balance in disbursement wallet",
"requestId": "6e3c1d3d-6b96-4e08-8a7d-c4968ea6905a",
"timestamp": "2026-07-03T14:30:05+03:00",
"data": null
}

```

11. Rate limits

Endpoint type	Default limit
Payment creation	60 requests / minute per `client_id`
Status queries	120 requests / minute per `client_id`

Exceeded limits return HTTP 429 with a Retry-After header.

12. Security best practices

1. **Never expose** `client_secret` in client-side code or mobile apps 2. All API calls must originate from your **backend server** 3. Use HTTPS for all communication 4. Verify every webhook signature before updating order status 5. Treat `transactionId` as the authoritative payment identifier — not your reference alone 6. Implement idempotent callback handling (same `transactionId` may arrive more than once) 7. Log `requestId` and `transactionId` for support and reconciliation 8. Rotate credentials through Lipahuru if compromised

13. MSISDN format

Always use international format without +:

```

255754123456
└─┬ 9-digit subscriber number
  └─ Tanzania country code

```

Valid pattern: 255 followed by exactly 9 digits.

14. Support

For integration support, credential requests, or go-live approval:

- **Email:** support@lipahuru.com *(update with your support contact)*
- **Include in support requests:** `requestId`, `transactionId`, `reference`, timestamp, and error code

Appendix A — Quick reference

Action	Method	Path
Get token	POST	`/oauth/token`
C2B collection	POST	`/api/v1/payments/collections/push`
B2C disbursement	POST	`/api/v1/payments/disbursements`
Query status	GET	`/api/v1/payments/{transactionId}`
Transaction history	GET	`/api/v1/transactions`
Refund	POST	`/api/v1/payments/{transactionId}/refunds`

Wallet balances	GET	<code>`/api/v1/wallets`</code>
Wallet by network	GET	<code>`/api/v1/wallets/{providerCode}`</code>

Appendix B — Test credentials

UAT credentials are issued separately after merchant onboarding. Contact Lipahuru to request UAT access.

Use UAT base URL and UAT callback URL during development. Never use production credentials in test environments.